

The Maths Behind Normalization

How this platform puts judges who score on different personal scales onto one common scale — stated exactly, then proved on the real `fixtures.json` data.

1. The problem, measured

`fixtures.json` holds 126 evaluations of 41 projects by 30 judges, each scored 2-5 on functionality / quality / innovation. The judges do not share a scale:

judge	mean given	spread (sd)	evaluations
jdg_01	2.00	0.00	1
jdg_14	3.00	0.67	3
jdg_27	3.00	0.47	2
...	...	...	...
jdg_15	4.06	0.83	6
jdg_30	4.08	0.57	4
jdg_02	4.22	0.81	6

The strictest judge averages **2.00**, the most lenient **4.22** — a 2.2-point gap on a 4-point scale. A raw average confounds *project quality* with *who happened to judge it*: a project seen only by harsh judges looks bad, one seen by two lenient judges looks elite. That is the entire problem normalization has to solve, and averaging cannot solve it by construction.

2. The method (what the code actually does)

Normalization here is not one trick but five stages, each removing one way a judge's personal scale can leak into the ranking. Code pointers are exact.

Stage 1 — Criterion averaging (`score_evaluation`, `hier_score.py`)

One evaluation's criterion scores become a single number (mean under equal weights here — `fixtures` define no weights, so shares are uniform; organizer weights are normalized to sum 1 in production). Per-criterion scales (`score_lo/score_hi`, migration 0016) map every criterion to 0-1 first, so a 1-5 rubric and a 0-10 rubric are comparable before weighting.

Stage 2 — Pairwise conversion (`generate_pairs`, `pairwise.py`)

Each judge's scores become **within-judge pairwise orderings**: for every pair of projects a judge scored, the higher-scored one wins. This is where absolute scale dies — a judge scoring (2, 3, 4) and one scoring (60, 70, 80) produce the *identical* evidence. Three rules keep it honest:

- **Unit weights, always.** Every comparison carries weight 1.0, enforced by a database CHECK constraint as well as in code. Score *margins* never become

evidence weights — otherwise a judge using a wide scale would outvote a careful one purely for being loud.

- **Ties produce nothing.** Scores within 1e-9 emit no observation (the epsilon exists because weights like 0.35 aren't binary-exact; crowning a winner on float dust would be fabrication).
- **Snapshots, not live reads.** Pairs are built from stored weighted scores, so later rubric edits can never reinterpret history.

Stage 3 — Reliability-weighted MAP fit (`fit`, `crowd_bt.py`)

Model: $P_j(A > B) = \text{sigmoid}(r_j \cdot (\theta_A - \theta_B))$, judge reliability $r_j = \exp(\log r_j)$, priors $\theta \sim N(0, 2^2)$, $\log r_j \sim N(\mu_j, \sigma_j^2)$ with cross-event historical or neutral $(0, 0.60)$ priors. Fitted by BFGS from zeros with an analytic gradient.

Two normalizing effects live here. First, **reliability pools agreement, not scale**: a judge whose orderings agree with the consensus gets high r regardless of whether they score 1-5 or 0-100; a contrarian's r collapses toward 0 and their pairs stop moving the ranking (demonstrated adversarially in `scripts/validate_ranking.py` experiment 3: inverted judge $\rightarrow r \approx 0.36$ vs ≈ 1.97 for consistent judges). Second, **the 0 prior shrinks thin evidence**: a project with two evaluations cannot stray far from 0 no matter its raw scores — absence of evidence is reported as modesty, not confidence.

Stage 4 — Sum-to-zero reporting (`crowd_bt.py`, `rank`)

The BT likelihood depends only on *differences*, so reported thetas are centered (reference project included in the mean). Rankings, gaps and probabilities are therefore symmetric and stable across recalculations.

Stage 5 — Laplace uncertainty (same file)

The full inverse-Hessian theta block propagates through the centering shift — including the reference project, which is no longer special after Stage 4 — giving every project an honest marginal std, pairwise $P(A>B)$ flags, and High/Low confidence. Stated assumption: thetas are treated as independent Gaussians for pairwise probabilities (posterior correlations are dropped; useful as a flagging heuristic, not calibrated odds).

The second opinion: severity pooling (`hier_score.py`)

Where BT needs pairs, the edge-case Bayesian model ($R = \theta + b + \epsilon$, EM variance components, `v2`) estimates an explicit per-judge severity b_j with partial pooling — a judge seen once collapses to the population instead of absorbing project spread. It is a different estimator of the same correction, and §4 checks they agree on fixtures.

3. Theorem (affine invariance, sketch)

Claim. Let one judge’s scores undergo any strictly increasing affine map $x \mapsto ax + b$, $a > 0$. Then the fitted thetas are *exactly* unchanged.

Proof. (i) The map preserves strict order, so `generate_pairs` emits a byte-identical pair set (ties map to ties: $ax+b$ preserves equality too). (ii) The MAP objective depends on the data only through that pair set (unit weights, no margins anywhere), with identical priors, reference, and start point. (iii) BFGS is deterministic given identical objective/gradient/start, so the optimum — and hence every reported theta — is identical. ■

Corollaries a statistician should verify rather than take on faith: the claim covers affine maps only (a judge who *reorders* projects changes the evidence — correctly so); reliability posteriors are likewise invariant since they see the same pairs; the θ prior introduces mild shrinkage but no scale dependence.

4. Proof on `fixtures.json` (ran, not asserted)

Pipeline: 126 fixture scores $\rightarrow$ criterion means $\rightarrow$ 254 pairs over 41 projects $\rightarrow$ BFGS MAP (`crowd-bt-map-v3`, neutral priors, reference `prj_01`, converged in 51 iterations). Raw baseline: plain mean of criterion-means per project.

Raw ranking (naive mean):

#	project	raw mean	evaluations
1	prj_11	4.333	3
2	prj_34	4.333	3
3	prj_10	4.167	2
4	prj_25	4.111	3
5	prj_37	4.083	3

Normalized ranking (Crowd-BT θ):

#	project	θ	raw #
1	prj_34	+2.246	2
2	prj_37	+1.629	5
3	prj_07	+1.405	31
4	prj_11	+1.324	1
5	prj_25	+1.281	4

Bottom agrees too (BT: `prj_20`, `prj_05`, `prj_23`; raw: `prj_40`, `prj_05`, `prj_23`). Related but not identical — Kendall $\tau = \mathbf{0.563}$, top-5 overlap $\mathbf{4/5}$ — which is the point: raw means confound judge scale, BT removes it. Twenty-one of 41 projects move ≥ 5 places between the two rankings.

Worked example — `prj_07` (raw #31 $\rightarrow$ normalized #3). Its five scores:

judge	gave	judge's overall mean	reading
jdg_01	2.00	2.00 (only score ever given)	harsh judge, uninformative level
jdg_19	2.33	3.33	a full point below their norm — real signal <i>against</i> neutral
jdg_21	3.67	3.67	half point <i>above</i> their norm — signal <i>for</i>
jdg_12	4.00	3.50	a full point above their norm — strong signal <i>for</i>
jdg_26	4.67	3.70	

The raw mean (3.33) lets jdg_01's lone 2.00 and jdg_19's low score dominate. The pairwise view sees what matters: three judges placed prj_07 *above* peers they scored in the same sitting. Absolute levels cancel; relative placements accumulate. Conversely prj_10 (raw #3 → #16) rested on two evaluations, and the θ prior correctly refuses to crown thin evidence.

Affine-invariance check on the same data. Mapping jdg_08's scores 2-5 → 10-100 (30x - 50, strictly increasing): pair sets byte-identical, fitted thetas differ by **0.00e+00** (max over 41 projects), reliabilities unchanged to 4 decimals (jdg_08 1.1441, jdg_19 0.5648, jdg_12 1.0844). The wider scale bought jdg_08 exactly nothing — as §3 predicts.

Cross-model check. The hierarchical scorer independently recovers the same leniency ordering (most lenient: jdg_02, jdg_15, jdg_30 by both raw means and posterior severity b), with effects heavily shrunk (± 0.25) under thin replication — two different estimators, one agreed correction.

5. What normalization does NOT do (read before citing this doc)

- **No cross-component miracles.** Projects sharing no judge path sit on no common scale; the pipeline refuses to fit (connected_components gate) rather than invent a ranking. Normalization aligns scales, it does not create evidence.
- **Thin evidence stays modest.** Few evaluations → posterior hugs the prior; prj_10's demotion is the feature working, but it means a genuinely great project seen twice cannot outrank a good one seen ten times.
- **Reliability is agreement, not accuracy.** A panel of correlated-but-wrong judges (shared misconception, leaked rubric) would mutually reinforce. The adversary test covers inversion, not collusion.
- **Reference dependence is mild, not zero.** Sum-to-zero makes *reporting* shift-invariant, but the fitted optimum still feels the reference through the prior (re-demonstrated in context §42) — second-order, disclosed.

- **History transfer is mean-only.** Cross-event priors carry posterior means with neutral width (MAP has no posterior variance to propagate).

6. Reproduce it

inside the api container (offline-safe, numpy/scipy baked into the image):

```
docker compose exec -T api python3 - < /tmp/norm_probe.py # §4 tables
```

```
docker compose exec -T api python3 - < /tmp/norm_tail.py # invariance + severity
```

```
docker compose exec -T api python - < scripts/validate_ranking.py # 10/10 incl. scale-
```

Code: backend/app/modules/judging/{pairwise,crowd_bt,hier_score,reliability}.py.

Method reference: JUDGING.md (§model v3, Laplace, rubric scales, blend normalization), crowd_bt_architecture_poc_corrected.md (original design; note its §15/§16 magnitudes encode the removed margin weighting — evidence tables and orderings only). Doc versions: crowd-bt-map-v3, hier-bayes-score-v2.